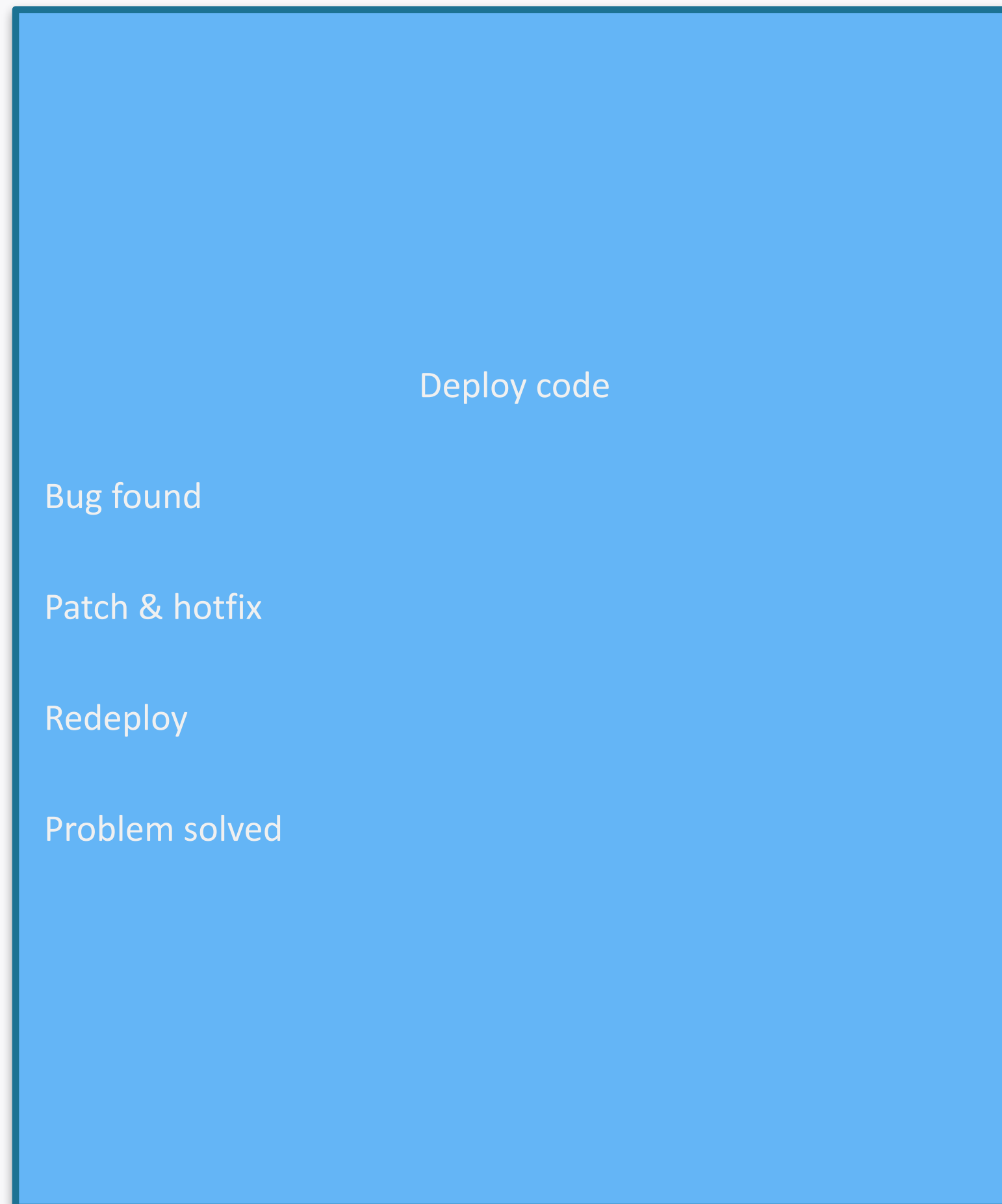


Module 6

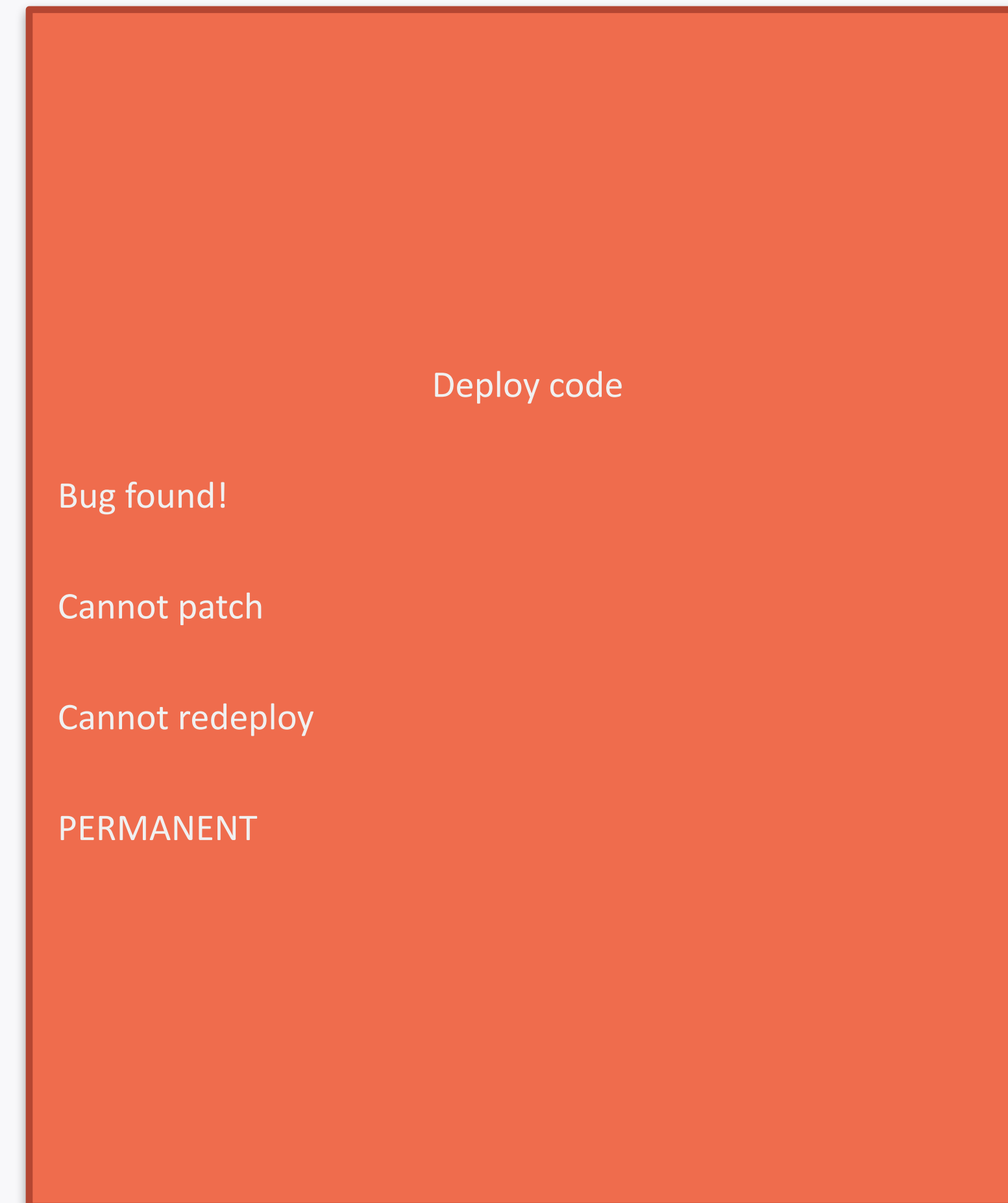
Smart Contract Security Fundamentals

Immutability = Higher Risk

Web2: Traditional Backend



Ethereum: Smart Contracts



Smart Contract Security Philosophy

Ethereum is Immutable

Code deployed = Code forever

No SSH access to servers

No hotfix capability

No database rollback

Core Principle:

"Ethereum does exactly what you
tell it to — not what you meant"

Therefore:

Security is foundational, not optional

Test before deploying

Assume worst-case scenarios

Reentrancy: The Famous Vulnerability

What Is Reentrancy?

Contract calls external address
External code calls back into
original contract BEFORE first
execution finishes

Real World Analogy:

ATM hands you cash, then
updates balance. If you could
interrupt in between, you could
request money again.

Result: Funds drained

Vulnerable Contract: Wrong Order

```
contract VulnerableVault {  
    mapping(address => uint)  
        public balances;  
  
    function withdraw(uint amt)  
        public {  
        require(balances[msg.sender]  
            >= amt);  
  
        // INTERACTION first!  
        msg.sender.call  
            {value: amt}("");  
  
        // Effect happens AFTER  
        balances[msg.sender] -= amt;  
    }  
}
```

Vulnerable pattern:

Interaction before effect

Attacker's fallback can
call withdraw() again

Balance hasn't decreased yet

Reentrancy Attack Step-by-Step

User calls `withdraw()`

Request \$100

Contract sends ETH

External call

Attacker fallback triggered

Calls `withdraw()`

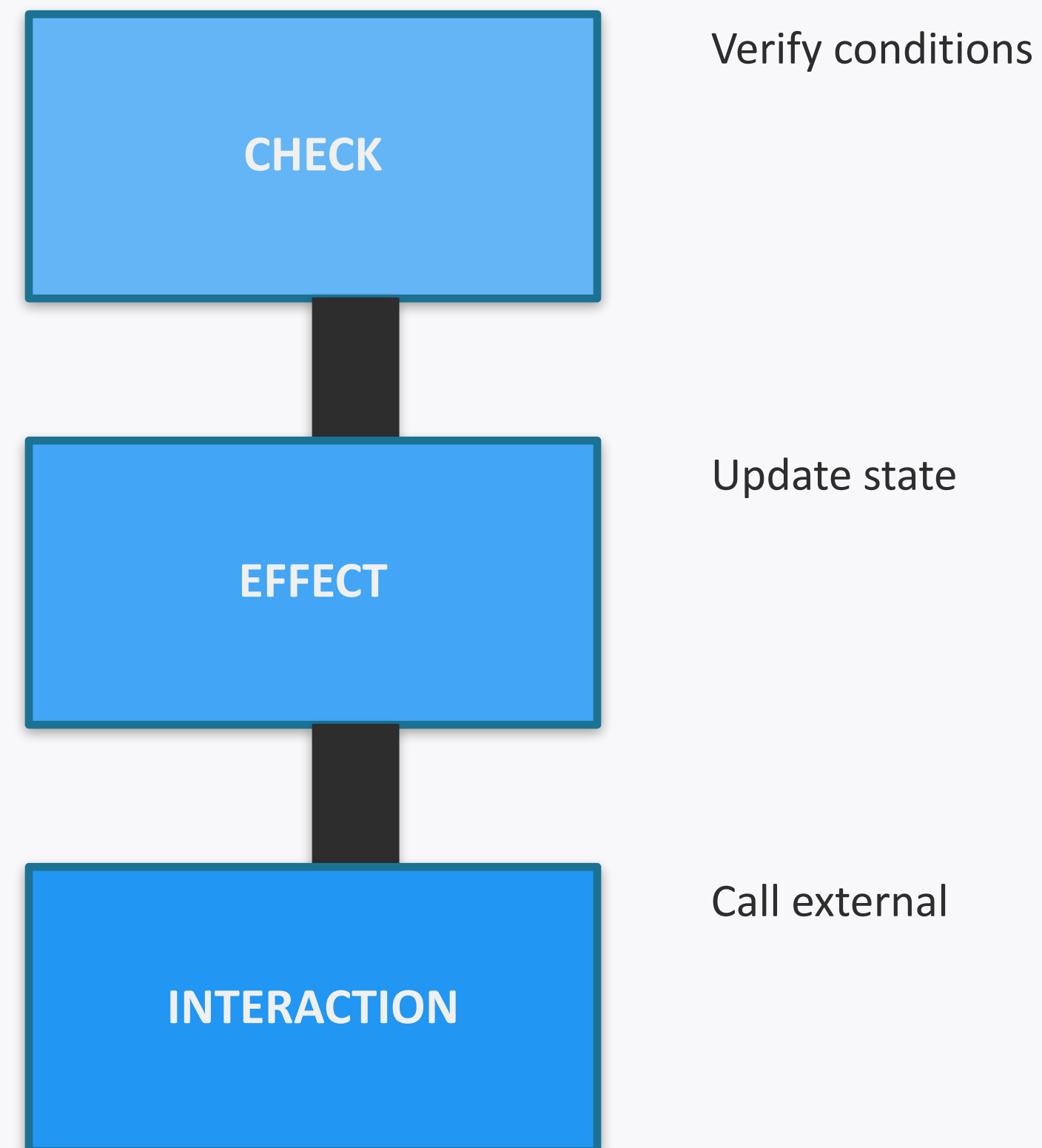
Balance not yet updated

Drain funds!

Original call continues

Too late...

The Fix: Checks-Effects-Interactions



Secure Contract: Correct Order

```
function withdraw(uint amt)
  public {
    // CHECK
    require(balances[msg.sender]
      >= amt);

    // EFFECT (first!)
    balances[msg.sender] -= amt;

    // INTERACTION (last!)
    msg.sender.call
      {value: amt}("");
  }
```

Secure pattern:

State updated BEFORE
external call

Even if reentry happens,
balance is already reduced

Access Control: Restricting Functions

```
contract Bank {
    address public owner;
    uint public funds;

    constructor() {
        owner = msg.sender;
    }

    modifier onlyOwner() {
        require(msg.sender == owner,
            "Not owner");
        _;
    }

    function setFunds(uint amt)
        public onlyOwner {
        funds = amt;
    }
}
```

Modifier restricts access

onlyOwner() ensures only
contract creator can
call setFunds()

Integer Overflow & Underflow

Before Solidity 0.8.0

`uint8 x = 255;`

`x = x + 1; // becomes 0!`

Silent wrapping (bad!)

Solidity 0.8+ Protects By Default

Overflow/underflow automatically

reverts

Exception: unchecked {} block

(use with caution)

Key Takeaway:

Modern Solidity is safer

But understand edge cases

Common Security Mistakes

Reentrancy Mistakes:

- Sending ETH before state update
- Not considering callback behaviour

Access Control Mistakes:

- Forgetting modifier on admin function
- Trusting frontend validation
- Assuming user honesty

General Mistakes:

- Assuming Ethereum won't call back
- Not testing edge cases
- Deploying without testing

Pre-Deployment Security Checklist

Before Deploying to Mainnet:

- ✓ All admin functions restricted?
- ✓ State updated before external calls?
- ✓ All inputs validated?
- ✓ Edge cases tested?
- ✓ Reentrancy protection in place?
- ✓ Gas limits considered?
- ✓ Events emitted for tracking?

Remember:

Once deployed, you cannot change it
Test thoroughly first

Key Security Patterns

1. Checks-Effects-Interactions (CEI)

Verify → Update state → External call

2. Access Control with Modifiers

```
restrict(msg.sender == owner)
```

3. Input Validation

```
require(amount > 0, "Invalid")
```

4. Event Emission

```
emit Transfer(from, to, amount)
```

5. Pull Pattern (not Push)

Let users withdraw, not auto-send

Why Immutability Changes Everything

Traditional Programming:

Bug found → Fix deployed → Issue resolved

Users might experience brief downtime

Smart Contract Programming:

Bug found → Cannot fix → Permanent

Funds may be lost forever

Reputation damage

This Means:

Testing is not optional

Code review is critical

Assume worst case scenarios

Exercise: Fix a Vulnerable Vault

Requirements:

- Accept deposits
- Allow withdrawals
- Prevent reentrancy
- Prevent Denial Of Service
- Prevent Locking the Contract

Key Concepts to Apply:

- CEI pattern for withdraw
- onlyOwner modifier
- State update before external call
- Event emission for tracking

Module 6 Recap: Security Fundamentals

- ✓ Immutability = No hotfixes
- ✓ Reentrancy: Interaction before effect
- ✓ CEI pattern: Check → Effect → Interact
- ✓ Access control via modifiers
- ✓ Solidity 0.8+ prevents overflow
- ✓ Test before deploying

Guiding Principle:

"Ethereum does exactly what you
tell it to — not what you meant"

Security is the foundation of

everything that follows